



The City College
of New York

CSC 59867-E: Senior Project II

AI Agents for Decision Making in the Real World

By Saptarashmi Bandyopadhyay

Email: sbandyopadhyay@ccny.cuny.edu

Assistant Professor of Computer Science

City College of New York and Graduate Center at the City University of New York

September 16, 2026 CSC 59867 - E



Where we left off

We established the communication penalty tradeoff and modeled link latency delays (processing, transmission, and queueing). We concluded by looking at how message pipelining segments packets to reduce store-and-forward delays across k links.

Today's Focus: We will expand this link-to-link model into full **Interconnection Topologies** to understand how physical architecture dictates routing bottlenecks, and then get into System-Level Parameters for Energy Utilization, Smart Grids, and Power Infrastructure.

Key Concepts:

- Topological bounds (Linear, Mesh, Hypercube) and collective communication primitives.
- Memory footprint optimization: Row vs. Column storage for matrix operations.
- **The Mathematical Goal:** Deriving the optimal processor cluster size to minimize total latency for distributed inner products.



Interconnection Topologies and Architectural Latency

The physical topology $G = (N, A)$ dictates routing paths and transmission bottlenecks:

- **Linear Array / Ring:** Diameter is $p - 1$ (array) or $\lfloor p/2 \rfloor$ (ring); node degree is at most 2. Latency scales as $O(p)$.
- **2D / d -Dimensional Mesh:** Diameter is $\Theta(p^{1/d})$; nearest-neighbor connectivity matches physical spatial discretizations.
- **Binary Balanced Tree:** Diameter is $\Theta(\log p)$; low link count ($p - 1$), but the root creates severe traffic congestion.
- **Hypercube (d -cube, $p = 2^d$):** Diameter is $d = \log_2 p$; node degree is $\log_2 p$; Hamming distance governs routing.



Collective Communication Primitives

- **Single Node Broadcast / Accumulation:** One processor transmits identical data to all nodes, or combines partial results via a reduction tree ($T_{\text{accum}} = T_{\text{bcast}}$ by time reversal).
- **Single Node Scatter / Gather:** One processor sends a distinct packet to every other processor, or collects distinct packets from them.
- **Multinode Broadcast / Accumulation:** Every processor simultaneously broadcasts a packet to all other processors.
- **Total Exchange:** Every processor sends a distinct, individual packet to every other processor.

Inherent Hierarchy: Total Exchange -> Multinode Broadcast -> Scatter/Gather -> Single Node Broadcast. An algorithm solving a higher-level primitive solves all descendants.



Node Degree & Cross-Section Bounds

Node Degree Bottleneck:

- For single node gather or multinode accumulation on a node of degree d , the node must receive $p - 1$ distinct packets.
- Theoretical lower bound:
$$\lceil (p - 1)/d \rceil$$

time units.

Cross-Section Bound (Total Exchange):

- Partition processors into disjoint subsets N_1 and N_2 separated by L_{12} physical links.
- In a total exchange, $|N_1| \cdot |N_2|$ packets must traverse these links:

$$T_{\text{total exchange}} \geq \max_{(N_1, N_2)} \frac{|N_1| \cdot |N_2|}{L_{12}}$$

- For a linear array: $L_{12} = 1 \implies \Omega(p^2)$ time.
- For a hypercube: $L_{12} = p/2 \implies \Omega(p)$ time.



Concurrency vs. Communication Penalty Tradeoffs

Increasing concurrency (adding processors p) reduces the computational subtask size per processor.

However, finer granularity requires more frequent message exchanges and synchronization checkpoints.

Packet Header Overhead Penalty:

- Transmitting small numerical payloads incurs fixed packet overhead (w bits).
- Effective bandwidth collapses when the payload size per message is small relative to the header overhead.

System Implication: Adding processors beyond a critical threshold increases CP and can degrade wall-clock completion time (S_p decreases).



Block Partitioning of Iterative Methods

Consider the vector iteration $x(t+1) = f(x(t))$ for $x \in \mathbb{R}^n$.

Instead of assigning one scalar coordinate per processor ($p = n$), partition the space into p blocks of size $k = n/p$

$$x_i(t+1) = f_i(x_1(t), \dots, x_p(t)), \quad i = 1, \dots, p$$

where each x_i is a k -dimensional block-component.

Computation per block per iteration: $\Theta(nk) = \Theta(n^2/p)$.

- Communication on a linear array (multinode broadcast of k coordinates): $\Theta(n)$
- Ratio:

$$\frac{T_{\text{COMM}}}{T_{\text{COMP}}} = \Theta\left(\frac{1}{k}\right)$$

As block size k increases, communication penalty becomes negligible, independent of total problem dimension n .



Row vs. Column Storage and Memory Footprint

Computing Ax where $A \in \mathbb{R}^{n \times n}$, with p processors ($k = n/p$ rows/columns per processor):

Row Storage Method:

- Processor i holds k rows of A and the entire vector $x \in \mathbb{R}^n$.
- Computes k local scalar products; requires a **multinode broadcast** of the results.
- *Memory footprint*: High. Each processor must replicate the full vector x .

Column Storage Method:

- Processor i holds k columns of A and only a k -dimensional subvector of x .
- Computes partial product vectors; requires a **multinode accumulation** across nodes.
- *Memory footprint*: Low. Local vector storage is strictly $O(k) = O(n/p)$.

Duality: For dense matrices, communication runtime is identical; column storage avoids memory inflation.



Analytical Latency Optimization in a Linear Processor Array

Let matrix-vector multiplication Ax be executed on a linear array of p processors with row storage ($k = n/p$).

Time Model Parameters:

- Floating-point arithmetic step (multiply-add): α
- Link transmission of a packet of size k : $\beta + k\gamma$ (where β is link setup/latency, γ is transfer time per scalar).

Total Execution Time per Iteration: $T(p) = \frac{\alpha n^2}{p} + (p-1) \left(\beta + \frac{n\gamma}{p} \right)$

Minimizing $T(p)$ with respect to p yields the optimal cluster allocation: $p^* \approx n \sqrt{\frac{\alpha - \gamma/n}{\beta}} \Rightarrow k^* = \frac{n}{p^*} \approx \sqrt{\frac{\beta}{\alpha}}$

Optimal subtask block size k^* is determined purely by the ratio of link latency β to floating-point compute speed α .



Example: Sizing Processor Clusters

- **Scenario:**

1. You need to compute a distributed inner product $c = \sum_{j=1}^n a_j b_j$ of dimension $n = 10^6$.
2. Processors are connected in a linear array; the result is accumulated at the central processor.
3. Scalar arithmetic time: $\alpha = 2 \text{ ns}$
4. Link communication cost per partial sum: $\beta = 198 \text{ ns}$

- **Task:**

1. Write down the total time function $T(p)$ as a function of p , accounting for both local partial-sum computation and tree/chain accumulation.
2. Analytically determine the optimal processor count p^* that minimizes total latency.
3. Compute the optimal block size $k^* = n/p^*$.



Solution (Part 1 - Derivation)

Analytical Derivation:

- Local computation per processor ($k = n/p$ terms - distribute the dimensions between all processors):

$$T_{\text{comp}} = k\alpha = \frac{n\alpha}{p}$$

- Accumulation along linear array to center node (distance $\lfloor p/2 \rfloor$ - see slide 3):

$$T_{\text{accum}} = (\alpha + \beta) \frac{p}{2}$$

- a. We need to communicate between nodes for each hop, and do another local computation at each step so that we can accumulate the total sum
- Total latency:

$$T(p) = \frac{n\alpha}{p} + \frac{(\alpha + \beta)p}{2}$$



Solution (Part 2 - Optimization)

Optimization: To find the optimal number of processors p^* , we simply need to treat the time function as a continuous function and find the critical point where $\frac{dT}{dp} = 0$

$$T(p) = \frac{n\alpha}{p} + \frac{(\alpha + \beta)p}{2}$$
$$\frac{dT}{dp} = -\frac{n\alpha}{p^2} + \frac{\alpha + \beta}{2} = 0 \implies p^* = \sqrt{\frac{2n\alpha}{\alpha + \beta}}$$

We can prove that this is the global minimum by showing that the second derivative is strictly positive for all $p > 0$ (we can't have a nonpositive number of processors):

$$\frac{d^2T}{dp^2} = \frac{2n\alpha}{p^3} > 0$$



Solution (Part 3.1 - Numerical Solution)

Numerical Solution:

We already know that $k^* = n/p^*$, so in order to find our solution, we need to solve for p^* using the analytical formula we just found:

$$p^* = \sqrt{\frac{2\alpha n}{\alpha + \beta}}$$

$$p^* = \sqrt{\frac{2 \cdot (2 \text{ ns}) \cdot (10^6)}{2 \text{ ns} + 198 \text{ ns}}} = \sqrt{\frac{4 \times 10^6}{200}} = \sqrt{20,000} \approx 141.42 \text{ processors}$$



Solution (Part 3.2 - Numerical Solution)

Numerical Solution:

Since we have two possible values (141 and 142), we need to plug them back in to find which solution truly gives us the lowest latency:

At $p = 141$:

$$T(141) = \frac{2 \times 10^6}{141} + \frac{200 \times 141}{2} \approx 14,184.40 + 14,100 = 28,284.40 \text{ ns}$$

At $p = 142$:

$$T(142) = \frac{2 \times 10^6}{142} + \frac{200 \times 142}{2} \approx 14,084.51 + 14,200 = 28,284.51 \text{ ns}$$



Solution (Part 3.2 - Numerical Solution)

Numerical Solution:

Since we have two possible values (141 and 142), we need to plug them back in to find which solution truly gives us the lowest latency:

At $p = 141$:

$$T(141) = \frac{2 \times 10^6}{141} + \frac{200 \times 141}{2} \approx 14,184.40 + 14,100 = 28,284.40 \text{ ns}$$

At $p = 142$:

$$T(142) = \frac{2 \times 10^6}{142} + \frac{200 \times 142}{2} \approx 14,084.51 + 14,200 = 28,284.51 \text{ ns}$$



Solution (Part 3.3 - Numerical Solution)

Numerical Solution:

Now that we know 141 processors gets us the lowest latency, we can calculate the optimal block size:

$$k^* = n/p^*$$
$$k^* = \frac{10^6}{141} \approx 7092 \text{ scalars per processor}$$

System-Level Parameters: Energy Utilization, Smart Grids, and Power Infrastructure

—

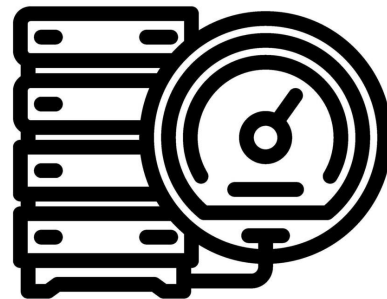
The interactions between AI and energy

Energy systems and AI interact in two ways: AI agents control complex electrical grids, while HPC clusters consume massive amounts of grid power

Grids must continuously balance volatile renewable generation with fluctuating customer demand to avoid blackouts.

Large clusters draw megawatts of concentrated power, requiring dedicated cooling and power management.

Today's Objective: Understand grid physics, formulate grid control as an RL problem, address distributed communication bottlenecks, and inspect datacenter power demands.



Real-World Hardware Example

Hardware Example (NVIDIA DGX B200):

- 8 Blackwell GPUs with 1.4 TB unified HBM3e memory.
- System power rating: up to ~ 14.3 kW per 10RU server chassis.
- Rack density has climbed from legacy 10 kW levels to 40–100 + kW per rack requiring direct-to-chip liquid cooling loops.

Power Usage Effectiveness (PUE):

$$\text{PUE} = \frac{\text{Total Facility Energy}}{\text{IT Compute Energy}}$$

A PUE of 1.15 means 15% of facility energy is consumed by pumps, chillers, and power distribution rather than computing.





Power Flow

Graph Representation: The transmission network is a graph $G = (V, E)$, where vertices V are substations (buses) and edges E are transmission lines and transformers.

Governing Physics: Unlike internet routing where packets choose arbitrary paths, electric current distributes across all connected lines according to Kirchhoff's and Ohm's laws.

Line Capacity Limits: Each transmission line (i, j) has a maximum thermal current rating $I_{ij}^{\max}$. Line utilization is defined as:

$$\rho_{ij} = \frac{|I_{ij}|}{I_{ij}^{\max}}$$

Cascading Blackouts: If $\rho_{ij} > 1.0$ for too long, safety breakers trip the line. The diverted power overloads adjacent lines, potentially triggering widespread grid collapse.



Grid Control as an RL Problem

Real-time telemetry including active and reactive generation, customer demand, branch current loads ρ_{ij} , and active line topologies.

Two Action Types ($\mathcal{A}$):

- Increasing or decreasing plant output (ΔP). This is effective, but expensive in fuel costs.
- Opening/closing switches and splitting substation buses. This redirects flow for free, but introduces a combinatorial action space ($> 10^4$ choices).

Reward Function ($\mathcal{R}$):

$$\mathcal{R}(s, a, s') = -C_{\text{operational}}(a) - \lambda \sum_{(i,j)} \max(0, \rho_{ij} - 1.0)^2 - \kappa \cdot \mathbb{I}_{\text{blackout}}(s')$$

The agent is penalized for operational costs, near-capacity line strain, and catastrophic blackouts.

Recall: Markov Decision Process (MDP)

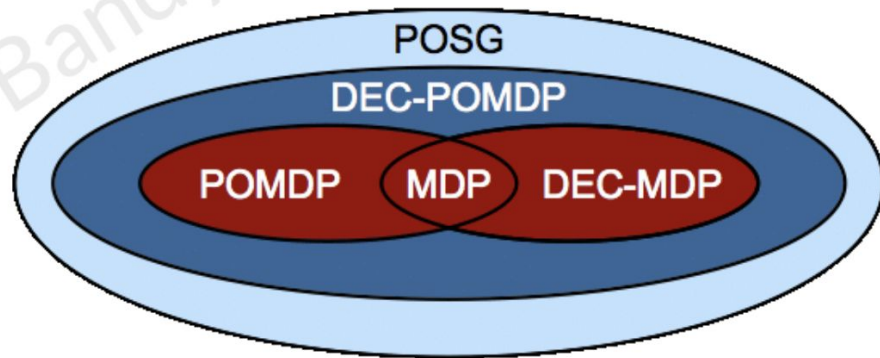
Agents: Single ($n = 1$).

Observability: Full (The agent knows the exact state s_t).

Objective: Maximize cumulative reward.

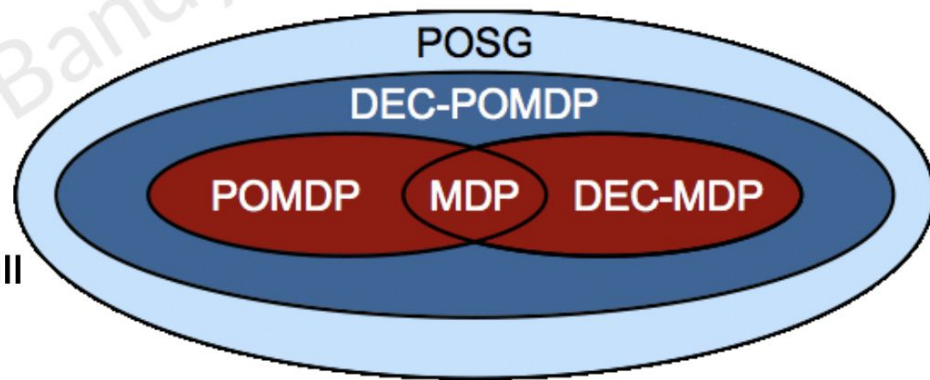
Tuple: $\langle S, A, T, R, \gamma \rangle$

- S : State space
- A : Action space
- T : Transition function
- R : Reward function



Recall: Decentralized MDP (DEC-MDP)

- **Agents:** Multiple ($n > 1$).
- **Decentralization:** Execution is decentralized (agents act independently), but planning can be centralized.
- **Observability:** Often implies **Joint Full Observability** (the combination of all agents' observations identifies the state), even if individual agents don't see everything.



- **Subset Relation:** $MDP \subset DEC - MDP$. (A DEC-MDP with 1 agent is an MDP).



Decentralized Control for Microgrids

Centralized control across an entire continent is too slow and creates a single point of failure.

With a DEC-POMDP, each local region or microgrid acts as an independent agent i that receives local observations o_i and selects actions a_i .

Recall: Non-Stationarity

- In Independent Q-Learning, other agents are constantly updating their policies.
- From agent i 's perspective, the environment transition probabilities keep shifting:

$$\mathcal{T}_i(s^{t+1}|s^t, a_i^t) \propto \sum_{a_{-i}} \mathcal{T}(s^{t+1}|s^t, \langle a_i^t, a_{-i} \rangle) \prod_{j \neq i} \pi_j(a_j|s^t)$$

This breaks the Markov property and can cause cyclical or unstable learning dynamics.



Parallel Methods for Power Flow

Calculating network voltages and power balance requires solving iterative systems of the form $x(t+1) = f(x(t))$

Every processor calculates its new value using the previous data in parallel, then waits at a synchronization barrier:

$$x_i(t+1) = f_i(x_1(t), \dots, x_n(t))$$

Gauss-Seidel Updates: Processors incorporate newly updated variables immediately, speeding up convergence but introducing serial dependencies:

$$x_i(t+1) = f_i(x_1(t+1), \dots, x_{i-1}(t+1), x_i(t), \dots, x_n(t))$$

Parallel Gauss-Seidel via Graph Coloring: Nodes that do not share a direct transmission line can update at the same time without data hazards, allowing parallel execution within color groups.



Grid2Op

Built by France's grid operator (RTE), the "Learning to Run a Power Network" challenge models weekly grid operations on standard IEEE test systems.

The key problem teams needed to address is that Agents face sudden renewable swings (10% to 30%+ energy penetration) and adversarial line outages.

Winning Strategies from NeurIPS:

- **Action Pruning:** Top teams reduced raw action spaces from 70,000+ down to $< 1,000$ viable candidate moves using expert rules.
- **Simulation-in-the-Loop:** Winning agents (such as Baidu's RLagent) ran prospective actions through a fast power-flow simulator before committing them to the physical grid.

Questions?

—